Lecture 30.06: AdaBoost (Implementation) — Scikit-Learn AdaBoostClassifier

Status Complete

Jupyter Notebook

Notes Hinglish

PDF English

PDF Hinglish

📌 Topic Overview

Production implementation of AdaBoostClassifier in Scikit-Learn. Demonstrates configuration of base estimators (DecisionTreeClassifier with max_depth=1), n_estimators tuning, learning_rate shrinkage, the SAMME.R real-probability algorithm, and evaluation across accuracy, precision, recall, and staged score convergence.

🗺️ Visual Architecture Diagram



🛠️ Mathematical Mechanics & Theoretical Foundations

In SAMME.R (real-probability AdaBoost), stump updates are weighted by log-ratio of predicted probabilities:

$$h_m^{(k)}(\mathbf{x}) = \ln p_k(\mathbf{x}) - \frac{1}{K}\sum_{j=1}^{K} \ln p_j(\mathbf{x})$$

💻 Python Production Implementation

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

def train_adaboost_pipeline(n_samples=500, n_features=6, n_estimators=50, learning
    """
    Builds, trains, and evaluates Scikit-Learn's AdaBoostClassifier.
    """
    # 1. Dataset generation
    X, y = make_classification(
        n_samples=n_samples, n_features=n_features, n_informative=4, random_state=
    )
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # 2. Configure AdaBoost with single-split Decision Stumps
    base_stump = DecisionTreeClassifier(max_depth=1, random_state=42)
    ada = AdaBoostClassifier(
        estimator=base_stump,
        n_estimators=n_estimators,
        learning_rate=learning_rate,
        random_state=42
    )
    ada.fit(X_train, y_train)

    # 3. Evaluate on test set
    y_pred = ada.predict(X_test)
    test_acc = accuracy_score(y_test, y_pred)

    # Staged scores convergence
    staged_accs = list(ada.staged_score(X_test, y_test))

    report = classification_report(y_test, y_pred, output_dict=True)

    return ada, test_acc, staged_accs, report

# ----------------- Driver Execution & Testing -----------------
if __name__ == "__main__":
    model, acc, staged, rep = train_adaboost_pipeline(n_estimators=50, learning_ra

    print("=" * 60)
    print("--- SCIKIT-LEARN ADABOOST PIPELINE EXECUTION ---")
    print(f"Final Test Accuracy (50 Stumps) : {acc * 100:.2f}%")
    print(f"Accuracy at Stage 1 (1 Stump)   : {staged[0] * 100:.2f}%")
    print(f"Accuracy at Stage 10 (10 Stumps): {staged[9] * 100:.2f}%")
    print(f"Accuracy at Stage 50 (50 Stumps): {staged[49] * 100:.2f}%")
    print(f"Precision (Class 1)             : {rep['1']['precision'] * 100:.2f}%")
```

```
    print(f"Recall (Class 1)               : {rep['1']['recall'] * 100:.2f}%")
    print("=" * 60)
```

🔑 Critical Architectural Takeaways

1. **Sequential Bias Minimization**: Unlike bagging which stabilizes variance, boosting models sequentially reduce bias by focusing on residuals and hard examples.
2. **Shrinkage Regularization**: Setting learning_rate < 1.0 enforces small gradient descent steps, preventing the ensemble from memorizing noise.
3. **Weak Learners Principle**: Keeping base decision trees shallow (max_depth=1 to 3) ensures individual estimators remain high-bias weak learners.